

Here is your step-by-step guide and implementation for **Stage 0**. Follow these instructions to set up your environment, verify your LLM connection, and complete your first checkpoint.

Step 1: Create the JOB-CARD.md

Create a file named `JOB-CARD.md` in your root directory. It serves as your structural target.

markdown

```
# Job Card: LLM Integration
```

```
## Target Output Structure
```

```
The model must output a valid JSON object matching the required schema.
```

```
## Rules
```

1. ****JSON Only****: The output must start with ``{`` and end with ``}``. No markdown formatting
2. ****Deterministic Fields****: All fields defined in the schema must exist, even if null or
3. ****No Conversational Filler****: The model must not include pleasantries, explanations, or

Use code with caution.

Step 2: Set Up Environment Variables

Prevent your API keys from leaking into version control by configuring your environment files first.

1. Create a `.gitignore` file in your root directory and add `.env`:

text

```
.env  
node_modules/
```

Use code with caution.

2. Create your actual `.env` file (replace with your real credentials):

env

```
LLM_BASE_URL="https://openrouter.ai/api/v1" # Or http://localhost:11434/v1/ for Ollama
LLM_API_KEY="your-actual-api-key-here"      # Use "ollama" if running locally
LLM_MODEL="google/gemma-3-1b-it:free"       # Or gemma3:1b for Ollama
```

Use code with caution.

3. Create your `.env.example` file (safe for Git):

env

```
LLM_BASE_URL=""
LLM_API_KEY=""
LLM_MODEL=""
```

Use code with caution.



Step 3: Write the Hello Script

Create the directory structure `src/llm/` and place the following code in `src/llm/hello.js`.

javascript

```
import OpenAI from "openai";

const client = new OpenAI({
  baseURL: process.env.LLM_BASE_URL,
  apiKey: process.env.LLM_API_KEY,
});

async function main() {
  try {
    const res = await client.chat.completions.create({
      model: process.env.LLM_MODEL,
      messages: [{ role: "user", content: "Reply with exactly the word: ready" }],
    });

    console.log(res.choices[0].message.content.trim());
  }
}
```

```
} catch (error) {  
  console.error("Error connecting to the LLM provider:", error.message);  
}  
}  
  
main();
```

Use code with caution.

Step 4: Update the README.md

Create a `README.md` file and add the required documentation about environment abstraction.

markdown

FlyRank Backend Track - Week 7 Assignment

LLM Provider Abstraction

By abstracting the provider configuration into three environment variables (``LLM_BASE_URL``

Use code with caution.

CHECKPOINT & COMMIT

1. Run the test command in your terminal:

bash

```
node --env-file=.env src/llm/hello.js
```

Use code with caution.

Verify that the output prints something containing `ready`.

2. Check Git Status to ensure your secret key file is hidden:

```
bash
```

```
git status
```

Use code with caution.

Verify that `.env` is **not** listed in the untracked files.

3. Save your progress with your first official commit:

```
bash
```

```
git add JOB-CARD.md .env.example src/llm/hello.js README.md .gitignore  
git commit -m "Stage 0: job card, provider working, key in .env"
```